Group 9: Vending Machine Simulator Design Document

Grewal Gaurav

Nwachukwu Obinna

Mugunthan Vaaman

Elashaal Musa

Aghbari Kareem

Powers Casey

February 28, 2026

1. Introduction & Overview

This document details the design of a vending machine simulation software created in C++ using object-oriented programming. The simulator replicates a vending machine with 10 rows and 8 columns, featuring two modes: vending and restocking. It includes a coin handling system and both manual and automatic simulation options. The simulator takes input from files for setup and uses pseudo-random numbers to make automatic simulations repeatable.

2. System Architecture & Class Design

The system is organised around 4 main classes: VendingMachine, ItemManager, CoinManager and Simulator.

VendingMachine

The VendingMachine class is the central controller. Key attributes include items: SpringCoil[10][8] , coinManager: CoinManager , dooropen: bool , selectedRow: int , selectedCol: int , selectionState: enum {NONE, ROW_SELECTED, COMPLETE}. VendingMachine manages a 10x8 array of SpringCoil objects (row A-J, columns 1-8), a total of 80 objects. It also handles the single object CoinManager which handles all monetary operations. SelectionState tracks item selection and progresses from no selection, to a row selection (A-J), to a specific column on the row (1-8) when both buttons are pressed in succession. The class maintains the current operating mode through a boolean doorOpen flag, Vending mode vendingMode() when the door is closed and restock mode restockMode() when the door is open.

In vending mode, CoinManager accepts or rejects coins and updates the balance. pressButtonLetter(row: char) and pressButtonNumber(col: int) record the customer's selection and trigger tryDispense(). The pressCoinReturn() instructs CoinManager to refund the customer's current balance. The tryDispense() member function contains the core logic of purchase to dispense. It checks whether the selected coil had stock, compares the customer balance to the price and calculates the change. Then decides whether to complete the sale or display the appropriate error message and resets the selection.

In restock mode VendingMachine allow the modification of internal states: setPrice(row, col, price) to update item’s price, refillCoil (row, col, quantity) updates the number of items in a coil, emptyCashBox() instructs CoinManager to calculate total earning since last reset and returns state to zero, refillChangeBox(denomination, count) updates the amount and denomination of coins in the changebox.

SpringCoil

SpringCoil represents a single coil position in the vending machine. Each coil contains one product type with 4 states: item name itemName: string , item price in cents price: int to avoid rounding errors, number of items remaining itemCount: int , maximum capacity per coil maxCapacity: int which is set to a fixed value 10.

When all the conditions are met to dispense by tryDispense , the SpringCoil activates dispense(): bool, which updates itemCount subtracting the dispensed amount from the current item count. dispense() returns true if the dispense amount is lower than or equal to the itemCount or false if it is higher. refill(amount: int) member function adds items up to the maximum capacity of 10. The following member functions are used to query coil state during a purchase process price getprice() ,item count getItemCount(), getItemname() ,

CoinManager

3. Flow Diagrams

Customer Vending Flow
Shows how a customer interacts with the machine. This includes inserting coins, selecting an item, and either receiving their item with change, getting an error message, or cancelling and getting their coins back.

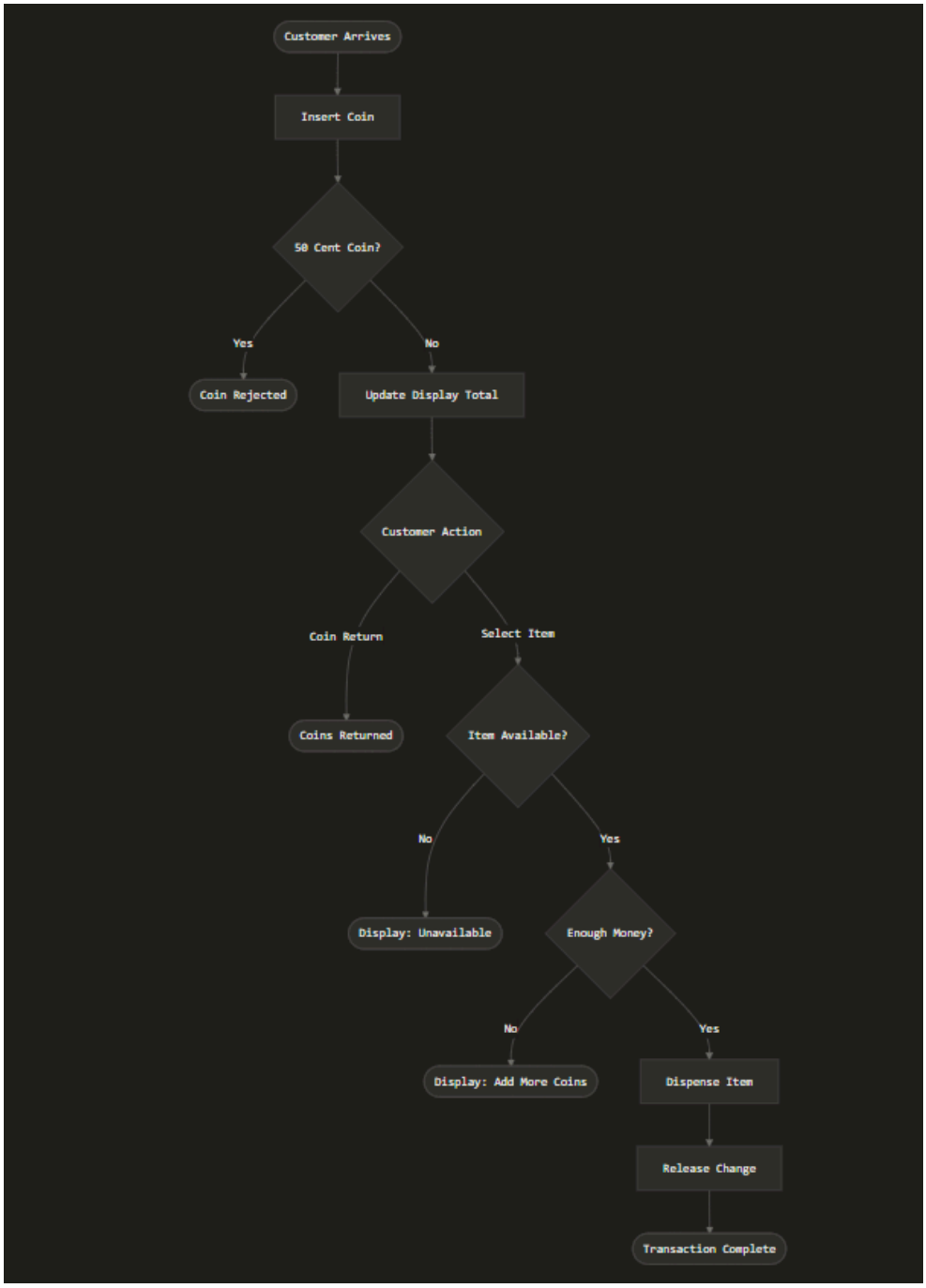

Fig 1. Customer Vending Flow

Service Worker Restock Flow

Shows how a service worker restocks the machine when the door is open. This includes setting prices, refilling coils, managing the cash and change boxes, then closing the door to return the machine to vending mode.

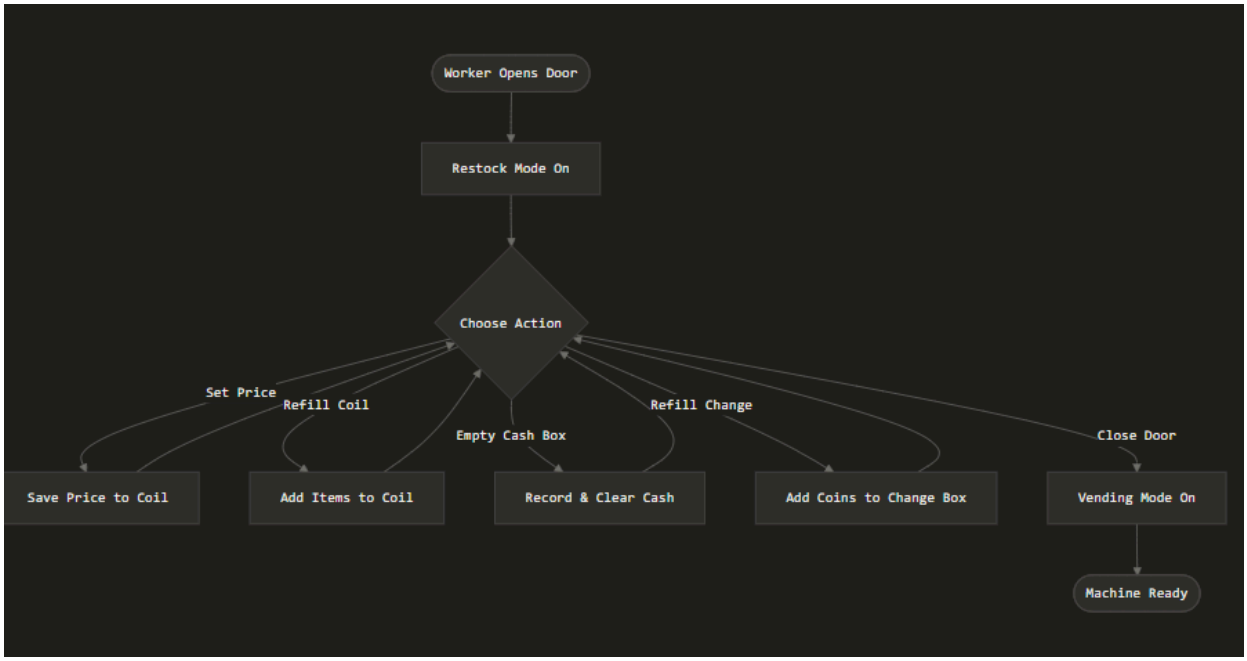

Fig 2. Restock Flow

Automatic Simulation Flow

Shows how the simulation automatically generates random customer and worker arrivals, runs them through their respective flows, logs the results to statistics, and repeats until the simulation ends and a final report is printed.

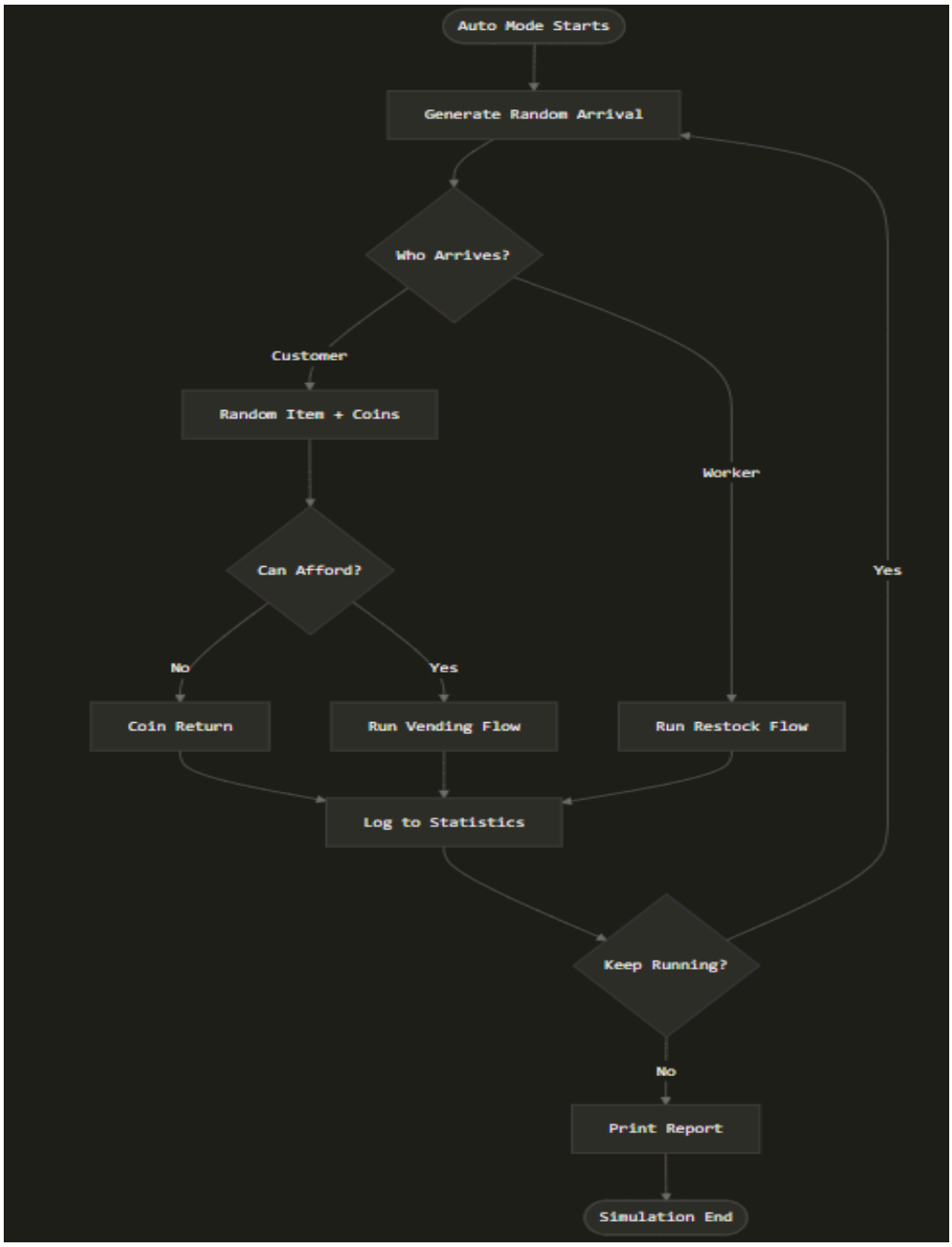

Fig 3. Simulation Flow

4. Algorithm Descriptions

Coin Insertion Algorithm

When a customer inserts a coin, the machine first checks if the door is open. If so, the coin is immediately rejected, else the coin's type is identified. If it is a 50 cent coin, it is rejected and

returned to the user using the coin return slot. Else, the coin's value is added to the customer's current running total and the display is updated in real time.

Item Selection Algorithm

When a customer presses a row button (A-J) followed by a column button (1-8), the machine records the selection and displays it. To enforce correct inputs, this is put in a while loop until the expected and allowed inputs are given. The machine then checks the corresponding coil. If it is empty, a "item unavailable" message is shown and no further action is taken. If the item is available, it checks whether the customer's entered total is greater than or equal to the item's price. If not, an "insufficient funds" message is displayed and the system goes back to allowing more coins. Finally, if both tests pass,the machine dispenses the item and calculates the change.

Dispensing and Change Algorithm

Once a valid purchase is confirmed, the spring coil releases the front item and the item count is decremented by one. The machine then calculates the change owing as the difference between the amount entered and the item's price. Starting from the largest coin denomination down to the smallest (toonie, loonie, quarter, dime, nickel), the machine checks how many of each coin are available in the change box and uses as many as needed without exceeding the change owing. Each coin directed out of the change box is dropped into the coin return slot. The customer's running total is then reset to zero. If the change box cannot make an exact change, this case is handled by the low-change check described below.

Coin Return Algorithm

When the customer presses the coin return button, the machine takes all coins currently held from the current transaction (those not yet dropped into the cash box) and routes them back through the coin return slot. The customer's running total is reset to zero and the display returns to its idle state.

Low Change Detection Algorithm

After every transaction, the machine evaluates the total remaining value in the change box. If the value falls below a defined threshold, the display shows an "exact change only" message. This check is also run at startup. When exact change only mode is active, the machine will only complete a transaction if the amount entered is exactly equal to the item's price, ensuring no change is needed.

Automatic Simulation Algorithm

When the simulation is set to automatic mode, the controller uses a random number generator to determine when the next customer arrives, based on a configurable average arrival rate. When a customer arrives, they randomly select a row and column, then insert randomly generated coin combinations up to a maximum spending threshold. If the item is too expensive

relative to what they are willing to spend, they press coin return and leave. Otherwise they complete the purchase. Separately, a restocking worker arrives at random intervals, opens the door, refills all coils that are below a threshold, replenishes the change box, empties the cash box, and closes the door. After each simulated event, statistics are updated.

Statistics Tracking Algorithm

Throughout the simulation, the statistics module records the following after each relevant event: total revenue collected, number of items dispensed per coil, total change dispensed, number of times "exact change only" was triggered, number of times a selection was unavailable, and number of times a customer had insufficient funds. On request, the statistics module formats and prints a summary report of all recorded values, which can be used to analyze machine performance and determine appropriate starting change box levels.

5. Key Assumptions & Design Decisions

Key Assumptions

- 50-cent coins are detectable and always rejected. All other accepted types are recognized automatically.
- Change is made using the largest coin type first. If exact change cannot be made, the transaction is blocked.
- Each coil holds a maximum of 10 items, all at the same price.
- Only one person uses the machine at a time. Vending is fully disabled when the door is open.
- In simulation mode, customers have a random spending limit and leave if the item exceeds it. Workers always fully restock when they arrive.

Design Decisions

- Integer currency — all values stored in cents to avoid floating point errors.
- Two modes — vending and restock mode, rather than checking door state on every action.
- Change algorithm — largest type first to minimize coins dispensed.
- Simulation is separate from machine logic — the machine behaves identically whether driven by a real user or the simulator (VendingMachine class has no knowledge of whether it's being used by a real user or a simulator.)
- Statistics tracked separately — keeps reporting concerns out of the core machine logic.

6. Libraries and Tools

C++ — required language for the project, and well suited for OOP with full support for classes, inheritance, and encapsulation.

<cstdlib> / <ctime> — built-in C++ libraries used to seed and generate random numbers for simulating customer arrivals and behaviour in automatic mode.

<iostream> — standard input/output library for reading user input and printing the machine display to the console in manual mode.

<string> — standard library for handling text output such as display messages and error prompts.

<vector> — standard library for managing dynamic collections such as the list of items in a coil or coins in the change box.

Git / GitHub — version control for managing collaboration and tracking changes between team members.

7. Conclusion

This report has outlined the design of a vending machine simulator in C++, covering the system's class structure, algorithm logic, coin handling, and simulation behaviour.

The simulator is self-contained, relying only on the C++ standard library with no external dependencies. Both manual and automatic modes are supported through the same core machine logic, with the simulation layer operating independently on top of it.

The design decisions made throughout this report reflect considerations that mirror how a real vending machine would behave. Overall, the system is well scoped for implementation and provides a solid foundation to build from as coding begins.